Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп’ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №5

з дисципліни „ Професійний Python”

на тему „ Обробка помилок, файли та серіалізація на прикладі потокового імпорту спортивних результатів.”

Виконала:

студентка 3 курсу

групи ФеП-32

Клімащук Вероніка Ігорівна

Перевірила:

Доц. Куньо Іван Михайлович

Львів – 2026

1. Тема, мета та постановка задачі

Тема: обробка помилок, файли та серіалізація на прикладі потокового імпорту спортивних результатів. Мета — застосувати exceptions, context managers, pathlib, logging, CSV/JSON/YAML, streaming I/O, перевірку зовнішніх даних і атомарне збереження.

Індивідуальний варіант №9 узято з лабораторної роботи №3. Запис має містити athlete, sport, result, category, timestamp. Потрібно потоково читати CSV, перевіряти значення, фільтрувати записи, рахувати середнє та найкраще значення для порівнюваних результатів, підтримувати tolerant/strict режими, вести статистику, зберігати JSON і звіт помилок.

2. Схема вхідних даних

Поле	Тип після розбору	Перевірка
athlete	str	непорожнє ім’я спортсмена
sport	str	має входити до allowed_sports
result	float	число, скінченне, невід’ємне та в налаштованому діапазоні
category	str	має входити до allowed_categories
timestamp	datetime	відповідає timestamp_format

CSV

```
athlete,sport,result,category,timestamp
Olena Koval,running,12.4,senior,2026-10-20T09:15:00
Danylo Hrytsenko,running,13.0,senior,2026-10-20T09:30:00
Maksym Shevchenko,swimming,58.2,junior,2026-10-20T10:00:00
Iryna Bondar,cycling,42.8,masters,2026-10-20T11:30:00
Andrii Lytvyn,running,wrong,senior,2026-10-20T12:00:00
,swimming,62.1,junior,2026-10-20T12:30:00
Sofiia Melnyk,swimming,60.5,open,2026-10-20T13:00:00
Petro Novak,running,11.9,senior,2026-10-21T09:00:00
```

У демонстраційному наборі п’ять рядків валідні, три мають навмисні помилки: текст замість числа, порожнє ім’я та недозволена категорія.

3. Структура проєкту

```
SportsStreamingLab05/
├── pyproject.toml
├── README.md
├── config.yaml
├── experiments.py
├── data/sports_results.csv
├── output/sports_results.json
├── output/invalid_records.csv
├── logs/application.log
└── src/sports_streaming/
    ├── config.py
```

```
├── exceptions.py
├── exporters.py
├── logging_config.py
├── main.py
├── models.py
├── readers.py
├── services.py
├── validators.py
├── filters.py
├── analytics.py
└── batches.py
```

4. Ієрархія винятків та exception chaining

```
ApplicationError
├── ConfigurationError
└── DataError
    ├── DataImportError
    ├── DataExportError
    └── RecordValidationError
```

ConfigurationError позначає помилки конфігурації; DataImportError — помилки читання або заголовків CSV; RecordValidationError зберігає номер рядка й поле; DataExportError повідомляє про помилку атомарного запису. Винятки відокремлюють тип помилки від тексту повідомлення та дають main.py змогу завершити застосунок контрольовано.

Приклад chaining

```
try:
    result = float(row["result"])
except (KeyError, ValueError) as error:
    raise RecordValidationError(
        "Result must be numeric", line_number=line_number, field="result"
    ) from error
```

Для значення wrong первинною причиною є ValueError; RecordValidationError додає контекст предметної області. Оператор from error зберігає __cause__, тому traceback показує обидва рівні.

5. YAML configuration і її перевірка

```
input:
  path: data/sports_results.csv
output:
  path: output/sports_results.json
  error_path: output/invalid_records.csv
processing:
  mode: tolerant
  allowed_sports: [running, swimming, cycling]
  allowed_categories: [junior, senior, masters]
  result_rules:
    running: {min: 0, max: 100000}
    swimming: {min: 0, max: 100000}
    cycling: {min: 0, max: 100000}
  timestamp_format: "%Y-%m-%dT%H:%M:%S"
logging:
```

```
  level: INFO
  path: logs/application.log
```

config.py читає YAML через yaml.safe_load у блоці with. Після перетворення у незмінну AppConfig перевіряються режим, непорожні набори категорій і видів спорту, відповідність діапазонів, рівень logging та наявність вхідного файла. Неправильна структура перетворюється на ConfigurationError із збереженою причиною.

```
try:
    with path.open("r", encoding="utf-8") as stream:
        raw = yaml.safe_load(stream)
except (OSError, yaml.YAMLError) as error:
    raise ConfigurationError("Cannot load YAML configuration") from error
try:
    inp, out, proc, log = (raw[key] for key in
                           ("input", "output", "processing", "logging"))
    config = AppConfig(
        input_path=Path(inp["path"]), output_path=Path(out["path"]),
        error_path=Path(out["error_path"]), mode=str(proc["mode"]),
        allowed_sports=frozenset(proc["allowed_sports"]),
        allowed_categories=frozenset(proc["allowed_categories"]),
        result_rules={name: (float(v["min"]), float(v["max"]))
                      for name, v in proc["result_rules"].items()},
        timestamp_format=str(proc["timestamp_format"]),
        log_level=str(log["level"]).upper(), log_path=Path(log["path"]),
    )
except (KeyError, TypeError, ValueError) as error:
    raise ConfigurationError("Invalid configuration structure") from error
validate_config(config)
return config
def validate_config(config: AppConfig) -> None:
    if config.mode not in {"strict", "tolerant"}:
        raise ConfigurationError("processing.mode must be strict or tolerant")
    if set(config.result_rules) != set(config.allowed_sports):
        raise ConfigurationError("Every allowed sport must have one result rule")
    for minimum, maximum in config.result_rules.values():
        if minimum > maximum:
            raise ConfigurationError("Invalid result range")
    if not config.input_path.is_file():
        raise ConfigurationError(f"Input file does not exist: {config.input_path}")
```

6. Logging

logging_config.py налаштовує рівень іменований рівень із YAML, часову мітку, рівень повідомлення, назву модуля та текст. FileHandler пише UTF-8 журнал; StreamHandler дублює події в консоль. Записи не містять зайвих персональних даних спортсмена у повідомленнях про помилки.

```
logging.basicConfig(
    level=getattr(logging, level),
    format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
    handlers=[logging.FileHandler(path, encoding="utf-8"), logging.StreamHandler()],
    force=True,
)
```

```
2026-10-27 18:42:43 | INFO    | services | Sports results import/export started
2026-10-27 18:42:43 | WARNING | services | Invalid record at line 6 (result): Result must be numeric
2026-10-27 18:42:43 | WARNING | services | Invalid record at line 7 (athlete): Athlete name is required
2026-10-27 18:42:43 | WARNING | services | Invalid record at line 8 (category): Category is not allowed
2026-10-27 18:42:43 | INFO    | __main__ | Total=8 valid=5 invalid=3 exported=5
```

7. Streaming reader і валідація

```
try:
    result = float(row["result"])
except (KeyError, ValueError) as error:
    raise RecordValidationError("Result must be numeric", field="result") from error
if not math.isfinite(result) or result < 0:
    raise RecordValidationError("Result must be finite and non-negative")
with path.open("r", encoding="utf-8-sig", newline="") as stream:
    reader = csv.DictReader(stream)
    missing = REQUIRED - set(reader.fieldnames or ())
    if missing:
        raise DataImportError(f"CSV is missing columns: {', '.join(sorted(missing))}")
    for number, row in enumerate(reader, start=2):
        yield number, {key: value or "" for key, value in row.items()}
```

read_csv_rows повертає Iterator і видає по одному рядку. Файл відкритий через with протягом роботи генератора і закривається при завершенні або помилці. pandas чи list(reader) не використовуються. Парсер перевіряє кожне поле, а помилки перетворення float і datetime піднімає з chaining.

filters.py містить потокову фільтрацію; analytics.py реалізує top-N рейтинг через heapq.nsmallest із обмеженою пам’яттю та pairwise_changes через itertools.pairwise; batches.py формує пакети за допомогою itertools.islice. Послідовні результати порівнюються лише для однакового виду спорту й категорії.

8. Strict і tolerant політики та статистика

Tolerant-режим збільшує invalid, додає номер, поле і повідомлення у список помилок, пише warning і переходить до наступного рядка. Strict-режим виконує ті самі дії до першої помилки та повторно піднімає RecordValidationError. ImportStatistics зберігає total, valid, invalid, суми, кількість і мінімальні числові результати за кожним видом спорту окремо.

```
except RecordValidationError as error:
    stats.invalid += 1
    errors.append({"line_number": number, "field": error.field, "error": str(error)})
    logger.warning("Invalid record at line %d: %s", number, error)
    if config.mode == "strict":
        raise
    continue
stats.valid += 1
```

Режим	Переглянуто	Valid	Invalid	Результат
Strict	101 рядок до першої помилки	100	1	зупинився на фізичному рядку 102
Tolerant	1010	1000	10	продовжив обробку; yielded 1000

Strict доцільний для пакетів, де цілісність усього набору є обов’язковою. Tolerant підходить для імпорту зовнішніх наборів, у яких допустиме відхилення окремих рядків, за умови окремого журналу відхилень.

9. JSON serialization та atomic export

SportsResult — dataclass, яка перетворюється на dict, timestamp серіалізується у ISO 8601. JSON пишеться потоково як масив, без накопичення всіх об’єктів у списку.

```
with tempfile.NamedTemporaryFile(..., dir=path.parent, delete=False) as stream:
    temporary = Path(stream.name)
    writer(stream)          # серіалізація може перерватися
    stream.flush()
    os.fsync(stream.fileno())
temporary.replace(path)     # заміна після успішного завершення
```

Файл створюється тимчасово у тій самій директорії, очищується при будь-якому винятку й атомарно замінює ціль лише після успішного запису. Перевірка симульованого переривання підтвердила: попередній JSON залишився без змін, тимчасових файлів не було.

10. Власний context manager

```
@contextmanager
def logged_operation(name: str):
    logger.info("%s started", name)
    started = perf_counter()
    try:
        yield
    except Exception:
        logger.exception("%s failed", name)
        raise
    else:
        logger.info("%s completed in %.4f s", name, perf_counter() - started)
```

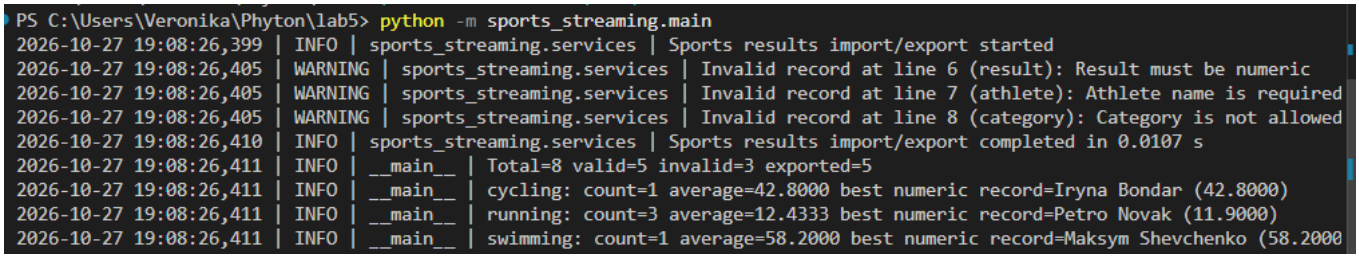
Генераторний менеджер з contextlib гарантує логування завершення або traceback помилки та не приховує виняток. Стандартні файлові ресурси також відкриваються оператором with.

11. Виконання програми

Команда запуску з кореня проєкту у PowerShell після встановлення залежності PyYAML:

```
$env:PYTHONPATH = "src"
python -m sports_streaming.main
```

Скриншот нижче відображає фактичний консольний запуск 27 жовтня 2026 року. Журнал і JSON створюються у logs/ та output/.



12. Експеримент eager проти streaming

Для кожного розміру окремо вимірювався час проходу CSV через csv.DictReader і пікове виділення Python-пам’яті через tracemalloc. Eager створював list(reader), streaming лише ітерував рядки. Показники залежать від системи, версії Python, диска і навантаження.

Записів	Eager час, с	Eager пік, MB	Streaming час, с	Streaming пік, MB
10 000	0.1092	4.276	0.0932	0.050
100 000	1.0536	42.549	0.8477	0.050
500 000	5.5682	213.224	4.4415	0.050

У цих вимірюваннях потоковий прохід мав майже сталий Python-пік близько 0,05 MB, тоді як eager зріс до 213,224 MB на 500 тис. рядків. На цій машині streaming також був швидшим, але така перевага часу не гарантована; ключова перевага — обмежене споживання пам’яті.

13. Додаткові перевірки помилок

Сценарій	Фактична поведінка
Відсутній config.yaml	ConfigurationError; причина FileNotFoundError
Неправильний YAML	ConfigurationError; причина yaml ParserError
Відсутній обов’язковий ключ	ConfigurationError; причина KeyError
Неправильний формат CSV/заголовок	DataImportError з повідомленням про поле/стовпці
Відсутній input-файл	ConfigurationError до початку імпорту
Збій під час JSON export	старий файл незмінний; тимчасовий файл прибраний

Перші три випадки виконані експериментальним скриптом. Наявність файлу перевіряється fail-fast у validate_config. CSV заголовок має окрему перевірку. Атомарність перевірена із контрольованим RuntimeError під час ітерації записів.

14. Відповіді на контрольні запитання

1. Що таке exception? Об’єкт Python, який сигналізує про помилку або іншу виняткову ситуацію.

2. Для чого використовується try? Для коду, у якому очікувано може виникнути виняток.

3. Для чого except? Щоб перехопити визначений тип винятку й виконати контрольовану реакцію.

4. Коли виконується else після try? Коли в блоці try не виникло винятку.

5. Коли виконується finally? Після try/except незалежно від успіху чи помилки.

6. Для чого використовується raise? Щоб явно підняти виняток.

7. Що означає raise без аргументів? Повторно підняти поточний виняток у блоці except.

8. Чому не рекомендується except: pass? Він приховує помилки та ускладнює діагностику.

9. Чому потрібно перехоплювати конкретні exception types? Так програма реагує лише на очікувані помилки й не маскує дефекти.

10. Що таке exception hierarchy? Ієрархія базових і спеціалізованих класів винятків.

11. Для чого створювати custom exceptions? Щоб надати помилкам зміст предметної області та зручно розрізняти їх.

12. Що таке exception chaining? Збереження причинного зв'язку між новим і первинним винятками.

13. Що означає raise ... from ...? Підняти новий виняток із явно заданою причиною.

14. Яку перевагу дає chaining для debugging? Traceback показує і технічну причину, і контекст прикладної помилки.

15. Що означає EAFP? Спробувати дію й обробити виняток, якщо вона не вдалася.

16. Що означає LBYL? Спершу перевірити умову, а потім виконати дію.

17. Що таке fail-fast? Раннє завершення на некоректній критичній умові.

18. Де доцільно застосовувати fail-fast? Під час стартової перевірки конфігурації та обов'язкових ресурсів.

19. Що таке context manager? Об'єкт або функція, яка керує початком і завершенням ресурсу/операції.

20. Для чого використовується with? Щоб гарантовано виконати завершальну дію при виході з блоку.

21. Які methods утворюють context manager protocol? __enter__() і __exit__().

22. Що робить __enter__()? Готує ресурс і повертає об'єкт для змінної після as.

23. Що робить __exit__()? Звільняє ресурс і вирішує, чи пригнічувати виняток.

24. Чи закриває with файл при exception? Так, файловий менеджер закриває його при виході з блоку.

25. Для чого contextlib.contextmanager? Для створення менеджера контексту з генераторної функції.

26. Яку роль має yield у generator-based context manager? Відділяє підготовку ресурсу від коду очищення після with.

27. Що таке pathlib.Path? Об'єктний інтерфейс Python для роботи зі шляхами файлової системи.

28. Чому pathlib кращий за ручну конкатенацію шляхів? Він портативно об’єднує компоненти й надає операції над шляхами.

29. Для чого задається encoding="utf-8"? Щоб читання й запис тексту не залежали від локального кодування ОС.

30. Що таке CSV? Текстовий табличний формат зі значеннями, розділеними роздільниками.

31. Для чого csv.reader? Для читання CSV як послідовності списків значень.

32. Для чого csv.DictReader? Для читання рядків CSV як словників, де ключі — заголовки.

33. Чим DictReader зручний для structured data? Звернення до полів іде за назвами, а не індексами стовпців.

34. Для чого csv.DictWriter? Для запису словників у CSV із заданими іменами стовпців.

35. Що таке JSON? Текстовий формат обміну вкладеними об’єктами та масивами.

36. Що таке serialization? Перетворення структури даних на формат для збереження або передавання.

37. Що таке deserialization? Відновлення структури даних із серіалізованого представлення.

38. Чим json.dump() відрізняється від json.dumps()? dump пише у файловий об’єкт; dumps повертає рядок.

39. Чим json.load() відрізняється від json.loads()? load читає з файлового об’єкта; loads розбирає рядок.

40. Що таке JSONDecodeError? Виняток стандартного модуля json для некоректного JSON.

41. Для чого використовується YAML? Для читабельних конфігурацій і структурованих даних.

42. Чому рекомендується yaml.safe_load()? Він не створює довільні Python-об’єкти з недовіреного YAML.

43. Які типові сценарії використання JSON? API, структурований експорт, обмін даними та кеші.

44. Які типові сценарії використання YAML? Ручне редагування налаштувань і конфігурацій.

45. Що таке configuration file? Зовнішній файл параметрів, які керують поведінкою програми.

46. Чому configuration краще не hard-code у Python? Параметри можна змінити без редагування логіки програми.

47. Що потрібно перевіряти у configuration? Типи й обов’язкові ключі, діапазони, режими, шляхи та сумісність параметрів.

48. Що таке logging? Система запису подій програми з рівнями й контекстом.

49. Чим logging відрізняється від print()? Logging має рівні, маршрутизацію, форматування й збереження подій.

50. Які існують logging levels? DEBUG, INFO, WARNING, ERROR і CRITICAL.

51. Для чого logger.info()? Для повідомлень про штатний перебіг програми.

52. Для чого logger.warning()? Для проблем, які не зупинили виконання, наприклад пропуску рядка.

53. Для чого logger.error()? Для помилки, що завадила операції або потребує реакції.

54. Для чого logger.exception()? Для запису повідомлення разом із traceback у except-блоці.

55. Що містить traceback? Стек викликів до помилки та інформацію про винятки й їхні причини.

56. Чому secrets не можна записувати в logs? Журнали можуть бути доступні ширшому колу й довго зберігатися.

57. Що таке streaming I/O? Обробка даних послідовними частинами без повного завантаження.

58. Яка перевага streaming для великих files? Пікова пам’ять не зростає пропорційно всьому набору записів.

59. Що таке eager file loading? Повне завантаження даних у пам’ять до їх обробки.

60. Чому list(reader) може бути небажаним? Він створює всі рядки одночасно й потребує значної пам’яті.

61. Що таке strict validation mode? Режим, у якому перший некоректний запис зупиняє імпорт.

62. Що таке tolerant mode? Режим, що реєструє та пропускає некоректні записи.

63. Коли доцільно використовувати strict mode? Коли кожен запис є обов’язковим для цілісності набору.

64. Коли tolerant mode? Коли можна імпортувати коректну частину з окремим звітом помилок.

65. Що таке atomic write? Запис у тимчасовий файл із заміною цілі лише після успіху.

66. Для чого використовується temporary file? Щоб незавершений результат не пошкодив наявний вихідний файл.

67. Що може статися при звичайному output write та аварійному завершенні? Цільовий файл може лишитися частково записаним або пошкодженим.

68. Навіщо видаляти temporary file після error? Щоб не залишати неповний артефакт і зайві файли.

69. Що таке JSON Lines? Формат, де кожен рядок містить окремий JSON-об’єкт.

70. Чому JSON Lines зручний для streaming? Можна читати й обробляти по одному об’єкту без розбору цілого масиву.

15. Висновок

Реалізовано надійний модуль для потокового імпорту спортивних результатів із CSV, перевірки за YAML-правилами, обробки помилок у двох режимах, журналювання та атомарного експорту JSON і окремого CSV відхилених записів. Exception chaining

зберігає першопричину, а менеджери контексту закривають ресурси та фіксують життєвий цикл операцій.

Експеримент із 500 000 записів показав на цій системі пікове використання Python-пам’яті 213,224 MB у eager-варіанті та 0,050 MB у потоковому. Strict/tolerant вимірювання підтвердили контрольовану зупинку й продовження обробки відповідно. Атомарний запис зберіг попередній файл після штучного збою. Отже, розроблений pipeline відповідає меті лабораторної роботи та продовжує предметну область варіанта №9.